

UNIT-III

Context Free Grammar (CFG) and Context Free Languages (CFL): Definition, Examples, Derivation, Derivation trees, Ambiguity in Grammar, Inherent ambiguity, Ambiguous to Unambiguous CFG, Useless symbols, Simplification of CFGs, Normal forms for CFGs: CNF and GNF, Closure properties of CFLs, Decision Properties of CFLs: Emptiness, Finiteness and Membership, Pumping lemma for CFLs.

//By Manish D@nge

What is Context-Free Grammar?

A grammar consists of one or more variables that represent classes of strings (i.e., languages) . There are rules that say how the strings in each class are constructed. The construction can use :

- Symbols of the alphabet
- Strings that are already known to be in one of the classes
- Or both

Context-Free Grammar

A context-free grammar (CFG) is a formal system used to describe a class of languages known as context-free languages (CFLs). Purpose of context-free grammar is:

- To list all strings in a language using a set of rules (production rules).
- It extends the capabilities of regular expressions and finite automata.

A CFG (or just a grammar) G is a tuple $G = (V, T, P, S)$ where

- V is the (finite) set of variables (or non terminals or syntactic categories). Each variable represents a language, i.e., a set of strings
- T is a finite set of terminals, i.e., the symbols that form the strings of the language being defined
- P is a set of production rules that represent the recursive definition of the language.
- S is the start symbol that represents the language being defined. Other variables represent auxiliary classes of strings that are used to define the language of the start symbol.

A grammar is said to be the Context-free grammar if every production is in the form of:

$G \rightarrow (VUT)^*$, where $G \in V$

- V (Variables/Non-terminals): These are symbols that can be replaced using production rules. They help in defining the structure of the grammar. Typically, non-terminals are represented by uppercase letters (e.g., S, A, B).

- *T (Terminals): These are symbols that appear in the final strings of the language and cannot be replaced further. They are usually represented by lowercase letters (e.g., a, b, c) or specific symbols.*
- *The left-hand side can only be a Variable, it cannot be a terminal.*
- *But on the right-hand side here it can be a Variable or Terminal or both combination of Variable and Terminal.*

The above equation states that every production which contains any combination of the 'V' variable or 'T' terminal is said to be a context-free grammar.

Core Concepts of CFGs

A CFG is defined by:

- Nonterminal symbols (variables): Represent abstract categories or placeholders (e.g., E, SE, S).
- Terminal symbols (alphabet): The actual characters or tokens in the language (e.g., a, b, +, *, (,) a, b, +, *, (,) a, b, +, *, (,)).
- Production rules: Specify how non terminals can be replaced with other non terminals or terminals (e.g., $E \rightarrow E+EE \rightarrow E + EE \rightarrow E+E$).
- Start symbol: A special nonterminal from which derivations begin.

CFG vs. Other Models

Model	Description
Finite Automata	Accept strings via computation (accept/reject).
<u>Regular Expressions</u>	Match strings by describing their structure.
CFG	Generate strings via recursive replacement.

Example: Arithmetic Expressions

Suppose we want to describe all legal arithmetic expressions using addition, subtraction, multiplication, and division.

Here is one possible :

Production Rules:

CFG:

$E \rightarrow int$

$E \rightarrow E Op E$

$E \rightarrow (E)$

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

$Op \rightarrow /$

Example Derivation:

E

$\Rightarrow E Op E$

$\Rightarrow E Op int$

$\Rightarrow int Op int$

$\Rightarrow int / int$

Various Properties of context free languages (CFL)

A Context-Free Language (CFL) is a formal language generated by a Context-Free Grammar (CFG) or Type 2 grammar (according to Chomsky classification). CFLs are accepted by Pushdown Automata and are essential in creating programming languages and building compilers.

Key Points:

- **Made with Grammar Rules:** A Context-Free Grammar ([CFG](#)) has rules that say how you can build valid sentences or expressions. Each rule replaces one symbol with a combination of other symbols.
- **Recognized by a Pushdown Automaton (PDA):** A CFL can be checked or accepted by a special machine called a Pushdown Automaton. It's like a basic computer that uses a stack (like a pile of plates) to keep track of what it's doing.
- **Can Handle Nested Structures:** CFLs are more powerful than [regular languages](#). They can describe things like matching parentheses (()), which regular languages can't handle properly.
- **Regularity:** context-free languages are Non-Regular PDA language.

Properties of CFL

1. Closure properties

The context-free languages are **closed** under some specific operation, closed means after doing that operation on a context-free language the resultant language will also be a context-free language. Some such operation are:

- Union Operation
- Concatenation
- Kleene closure
- Reversal operation
- Homomorphism
- Inverse Homomorphism

- Substitution
- prefix operation
- Quotient with regular language
- Cycle operation
- Union with regular language
- Intersection with regular language
- Difference with regular language

Context free language is **not closed** under some specific operation, not-closed means after doing that operation on a context-free language the resultant language not remains to be a context-free language anymore.

Some such operation are:

- Intersection
- Complement
- Subset
- Superset
- Infinite Union
- Difference, Symmetric difference (XOR, NAND, NOR or any other operation which get reduced to intersection and complement)

Read more about [Closure Properties of Context Free Languages](#)

2. Decision Properties

- **Emptiness:** We can decide if a CFL is empty (i.e., it has no strings).
- **Finiteness:** We can decide if a CFL has only a finite number of strings.
- **Membership:** We can decide if a string belongs to a CFL (using parsing techniques like [CYK algorithm](#)).
- **Equivalence:** Undecidable. We cannot always determine if two CFLs are exactly the same.
- **Universality ($L = \Sigma^*$) :** Undecidable.
- **Inclusion ($L_1 \subseteq L_2$) :** Undecidable.

3. Deterministic property

The context-free language can be:

- **DCFL-Deterministic** : These are the languages which can be recognized by deterministic pushdown automata.
- **NDCFL-Non-deterministic**: These are the context free languages that which can not be recognized by DPDA but can be recognized by NPDA context free language.

Ambiguous Grammar

Context-Free Grammars (CFGs) is a way to describe the structure of a language, such as the rules for building sentences in a language or programming code. These rules help define how different symbols can be combined to create valid strings (sequences of symbols).

CFGs can be divided into two types based on how they create derivation trees :

- **Ambiguous grammars:** These grammars can create more than one derivation tree for the same string. This means that a string can have different meanings or interpretations depending on how it's parsed.
- **Unambiguous grammars:** These grammars only allow one derivation tree for each string, so there's no confusion about how the string should be interpreted.

This classification helps us to understand how CFGs are used in language and computer systems to process information.

Ambiguous grammar

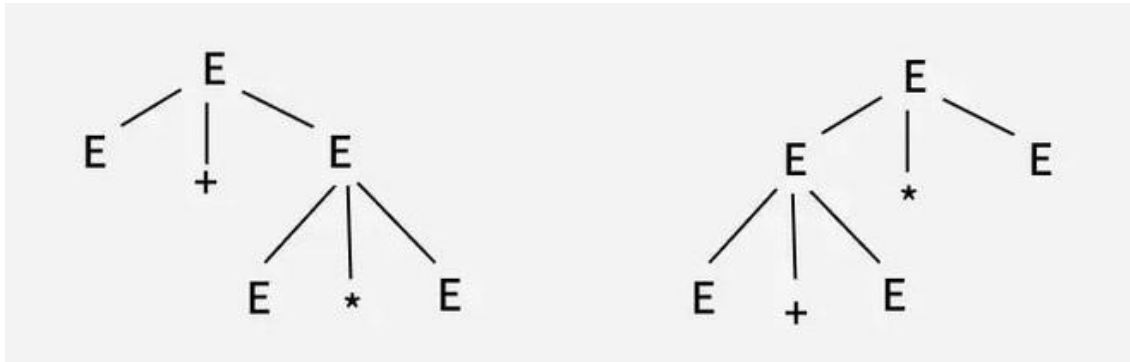
A [Context-Free Grammar](#) (CFG) is called **ambiguous** if there is a string that can have more than one valid derivation tree. This means the string can be generated in different ways, either through different LeftMost Derivations (LMDT) or RightMost Derivations (RMDT).

In simple terms, a CFG $G = (V, T, P, S)$ is ambiguous if there is a string in the terminal set T that can be produced in more than one way, creating multiple parse trees. Here:

- V is the set of variables (non-terminal symbols),
- T is the set of terminal symbols,
- P is the set of production rules,
- S is the start symbol, which is a special variable from where the derivation begins.

This happens when a string has multiple possible ways to be derived using the grammar rules, leading to more than one valid tree structure.

Example 1. Let us consider this grammar: $E \rightarrow E + E \mid E * E \mid id$, We can create 2 parse tree from this grammar to obtain a string **id + id * id**.



Both the above parse trees are derived from the same grammar rules but both parse trees are different. Hence the grammar is ambiguous.

Example 2. Let us now consider the following grammar:

Set of alphabets $\Sigma = \{0, \dots, 9, +, *, (,)\}$

$E \rightarrow I$

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$I \rightarrow ? \mid 0 \mid 1 \mid \dots \mid 9$

From the above grammar String **3*2+5** can be derived in 2 ways:

I) First leftmost derivation

$E \Rightarrow E * E$
 $\Rightarrow I * E$
 $\Rightarrow 3 * E + E$
 $\Rightarrow 3 * I + E$
 $\Rightarrow 3 * 2 + E$
 $\Rightarrow 3 * 2 + I$
 $\Rightarrow 3 * 2 + 5$

II) Second leftmost derivation

$E \Rightarrow E + E$
 $\Rightarrow E * E + E$
 $\Rightarrow I * E + E$
 $\Rightarrow 3 * E + E$
 $\Rightarrow 3 * I + E$
 $\Rightarrow 3 * 2 + I$
 $\Rightarrow 3 * 2 + 5$

Following are some examples of ambiguous grammar:

- $S \rightarrow aS \mid Sa \mid ?$
- $E \rightarrow E + E \mid E * E \mid id$
- $A \rightarrow AA \mid (A) \mid a$
- $S \rightarrow SS \mid AB, A \rightarrow Aa \mid a, B \rightarrow Bb \mid b$

Whereas following grammars are unambiguous:

- $S \rightarrow (L) \mid a, L \rightarrow LS \mid S$
- $S \rightarrow AA, A \rightarrow aA, A \rightarrow b$

Removal of Ambiguity in Grammar

To remove ambiguity from a grammar, follow these simple steps:

1. **Simplify production rules:** Break down complex rules into smaller and simpler ones. This way, the grammar won't allow multiple interpretations for the same string.
2. **Set precedence and associativity:** For things like math operations, make sure the order in which they are applied is clear. For example, define that multiplication happens before addition, or how to group operations (left-to-right or right-to-left).
3. **Fix left recursion:** Left recursion occurs when a rule refers to itself in a way that causes infinite loops. To fix it, change the rules so that recursion happens at the end, not at the start.
4. **Factor out common parts:** If two rules start the same way, combine the common part. For example, instead of having rules like $A \rightarrow \alpha\beta$ and $A \rightarrow \alpha\gamma$, make it $A \rightarrow \alpha A'$ and $A' \rightarrow \beta \mid \gamma$.

Inherent Ambiguity

A Context-Free Language (CFL) is said to be **inherently ambiguous** if every possible grammar for the language is ambiguous. This means no matter how you write the grammar for the language, there will always be strings in that language that can be parsed in more than one way.

Example:

Consider the language **L** defined as:

$$L = \{ a^n b^n c^m d^m : n \geq 1, m \geq 1 \} \cup \{ a^n b^m c^m d^n : n \geq 1, m \geq 1 \}$$

A grammar for this language could be:

- $S \rightarrow AB \mid C$
- $A \rightarrow aAb \mid ab$
- $B \rightarrow cBd \mid cd$
- $C \rightarrow aCd \mid aDd$
- $D \rightarrow bDc \mid bc$

Let's see how the string **aabbccdd** can be parsed. This string has two possible leftmost derivations:

Using $S \rightarrow AB$:

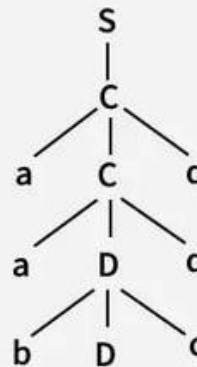
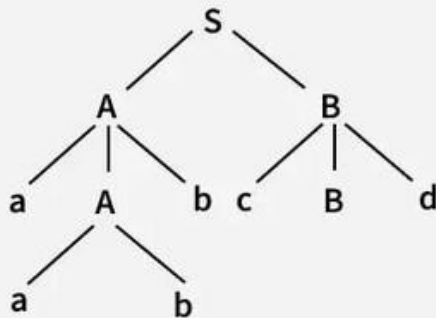
- $S \Rightarrow AB$
- $A \rightarrow aAb \Rightarrow aAbB$
- $A \rightarrow ab \Rightarrow aabbB$
- $B \rightarrow cBd \Rightarrow aabbcBd$
- $B \rightarrow cd \Rightarrow aabbccdd$

Using $S \rightarrow C$:

- $S \Rightarrow C$
- $C \rightarrow aCd \Rightarrow aCd$
- $C \rightarrow aDd \Rightarrow aaDdd$

- $D \rightarrow bDc \Rightarrow aabDcdd$
- $D \rightarrow bc \Rightarrow aabbccdd$

On parsing the string **aabbccdd** we get,



Both derivations result in the same string **aabbccdd**, but they come from different rules and paths, showing that there is more than one way to parse the string.

Since there are multiple ways to parse the string **aabbccdd** and this pattern holds for all grammars of **L**, we can conclude that **L** is inherently ambiguous. This means no grammar for **L** can avoid ambiguity for all strings in the language.

Important Points on Ambiguous Grammar

1) A grammar is **ambiguous** if it contains both **left recursion** and **right recursion**. This combination can lead to multiple ways of deriving the same string, creating more than one parse tree.

Example: $S \rightarrow SaS \mid \epsilon$

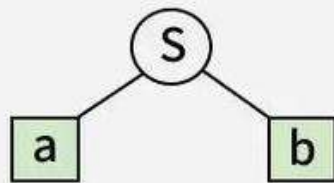
Here, the grammar has both **left recursion** ($S \rightarrow SaS$) and **right recursion** ($S \rightarrow \epsilon$), making it ambiguous. This allows multiple derivations for the same string, such as **{aa}**, leading to more than one parse tree.

2) Even if a grammar does not have both **left** and **right recursion**, it can still be **ambiguous**. The absence of recursion does not guarantee that the grammar is unambiguous.

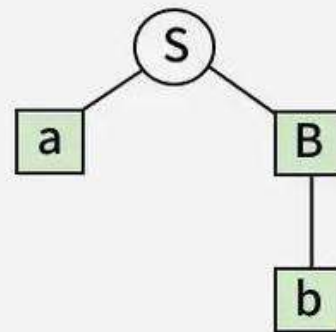
Example: $S \rightarrow aB \mid ab, A \rightarrow AB \mid a, B \rightarrow Abb \mid b$

In this example, there is no left or right recursion, but the grammar is still

ambiguous. For the string **{ab}**, we can derive it in multiple ways, resulting in more than one parse tree. This shows that even without recursion, a grammar can still be ambiguous.



Parse Tree 1



Parse Tree 2

Simplifying Context Free Grammars

A **Context-Free Grammar (CFG)** is a formal grammar that consists of a set of production rules used to generate strings in a language. However, many grammars contain redundant rules, unreachable symbols, or unnecessary complexities. **Simplifying a CFG** helps in reducing its size while preserving the generated language, making parsing more efficient.

Why Simplify a CFG?

Simplifying a CFG is essential for:

- **Efficient Parsing:** Smaller grammars require fewer computations.
- **Better Understanding:** Removing redundant parts makes it easier to analyze.
- **Optimization:** Improved computational efficiency in compilers and language processing.

Normal forms for CFGs

Normal forms for CFGs are standardized formats that simplify production rules while preserving the language generated by the grammar. The two most common normal forms are **Chomsky Normal Form (CNF)** and **Greibach Normal Form (GNF)**.

- **Chomsky Normal Form (CNF):** In CNF, each production rule adheres to one of the following formats: $A \rightarrow BC$, $A \rightarrow a$, or $S \rightarrow \epsilon$. CNF is particularly useful for algorithms like the CYK (Cocke–Younger–Kasami) parsing algorithm.

Chomsky Normal Form (CNF) is a canonical shape for context-free grammars that makes the grammar's structure regular and its parse trees uniformly binary. A grammar is in CNF if every production is one of three types: (1) a nonterminal produces exactly two nonterminals ($A \rightarrow BC$), (2) a nonterminal produces exactly one terminal ($A \rightarrow a$), or (3) the start symbol produces the empty string ($S \rightarrow \epsilon$) — and that last case is allowed only when the language contains ϵ . No other kinds of productions (mixed terminal/nonterminal right-sides, unit productions like $A \rightarrow B$, or longer right-sides) are permitted.

Intuition and significance

CNF forces every internal node in a parse tree to have exactly two children and every leaf to be a single terminal. That binary branching drastically simplifies reasoning about derivations: the structure of how substrings combine is fixed and local. This regularity is the basis for many theoretical results and algorithms: the CYK parsing algorithm, proofs of decidability and complexity for membership in CFLs, and combinatorial arguments (for example, in pumping and normal-form based proofs). Because any non-empty context-free language can be produced by some CNF grammar (except cautious handling of ϵ), CNF does not reduce expressive power — it trades syntactic freedom for analytic uniformity.

Conversion sketch (how to convert any CFG to CNF)

Converting an arbitrary CFG to an equivalent CNF grammar is a multi-step process that preserves the language (except for optional special handling of ϵ):

1. **Remove useless symbols** — eliminate nonterminals that never derive terminals or are unreachable from the start symbol.

2. **Eliminate ϵ -productions** — remove productions that derive the empty string while preserving derivations of nonempty strings; allow $S \rightarrow \epsilon$ only if needed.
3. **Remove unit productions** — replace rules of form $A \rightarrow B$ by copying B 's productions into A (transitively), then delete the unit rules.
4. **Convert mixed/long right-sides** — replace terminals appearing with nonterminals by new nonterminals ($X_a \rightarrow a$) and break right-sides longer than two into binary chains by introducing fresh nonterminals so that every rule is binary.

Each step can be carried out algorithmically and increases grammar size only polynomially.

Properties and consequences

- **Binary parse trees:** Every parse tree for a nonempty string becomes a full binary tree whose leaves are terminals.
- **CYK algorithm:** CNF enables the dynamic-programming CYK parser which decides membership in $O(n^3 \cdot |G|)$ time for input string length n .
- **Proof convenience:** Many proofs (complexity bounds, pumping lemmas, grammar transformations) rely on CNF because it localizes dependencies to two children.
- **ϵ and unit handling:** Care is needed to preserve ϵ and avoid introducing spurious empty derivations when removing ϵ -productions or unit rules.

Short example

If a grammar has $A \rightarrow a B C D$, CNF conversion introduces new nonterminals and rules like $A \rightarrow B X_1$, $X_1 \rightarrow C X_2$, $X_2 \rightarrow D X_3$, and $X_3 \rightarrow a$ (plus separate $X_a \rightarrow a$ if mixing terminals), thereby enforcing binary nonterminal productions and terminal isolation.

In summary, CNF is a mechanically achievable, powerful normal form that standardizes CFGs into a binary, tree-structured format that underpins parsing algorithms and theoretical analysis without changing the expressed language (aside from controlled handling of ϵ).

- **Greibach Normal Form (GNF):** In GNF, every production rule adheres to the format: $A \rightarrow a\alpha$, where A is a non-terminal, a is a terminal, and α is a string of non-terminals (or empty).

Greibach Normal Form (GNF) is a standardized representation of a context-free grammar (CFG) in which **every production begins with a terminal symbol**.

Formally, a grammar is in GNF if each rule has the form

$A \rightarrow a\alpha$, where A is a non-terminal, a is a terminal, and α is a (possibly empty) string of non-terminals. The only special exception is that the start symbol may produce the empty string $S \rightarrow \epsilon$ if and only if ϵ belongs to the language. No production in GNF can start with a non-terminal, and no production can have terminals appearing later on the right-hand side except the first symbol.

Core intuition

The deepest idea behind GNF is **immediacy of progress**. In an unrestricted CFG, a derivation can apply many rules without producing a single terminal symbol, which makes parsing and theoretical reasoning difficult. GNF removes this problem by forcing **every derivation step to generate exactly one terminal symbol at the front of the sentential form**.

As a result:

- Each derivation step corresponds directly to one terminal in the final string.
- A string of length n is derived in exactly n steps (ignoring ϵ).
- Infinite loops that delay terminal generation are structurally impossible.

Thus, GNF turns derivations into a **linear, forward-moving process**.

Structural meaning of GNF

In GNF, derivations follow a strict pattern:

1. Pick a non-terminal.
2. Replace it with a terminal followed by zero or more non-terminals.
3. Repeat until no non-terminals remain.

This means that:

- The **leftmost symbol is always a terminal** after each step.
- Parsing becomes predictive: the next terminal is known immediately.
- The grammar naturally supports **top-down (leftmost) parsing**.

From a tree perspective, GNF parse trees grow **one terminal at a time**, rather than branching first and filling terminals later.

Conversion to GNF (conceptual process)

Any context-free grammar can be converted into an equivalent GNF grammar (with careful ϵ handling). The transformation is more complex than CNF and proceeds roughly as follows:

1. **Remove useless symbols** – eliminate unreachable or non-generating variables.
2. **Eliminate ϵ -productions** – except possibly $S \rightarrow \epsilon$.
3. **Eliminate unit productions** – rules of the form $A \rightarrow B$.
4. **Remove left recursion** – both direct and indirect left recursion must be eliminated, since GNF cannot begin with a non-terminal.
5. **Order non-terminals** and substitute productions so that every rule begins with a terminal.
6. **Rewrite productions** until all rules follow $A \rightarrow a \alpha$ form.

Though technically involved, the conversion always terminates and preserves the language.

Important properties

- **Terminal-first rule:** Every production introduces a terminal immediately.
- **Linear derivation length:** Number of derivation steps equals string length.
- **No left recursion:** GNF grammars are guaranteed to be left-recursion free.

- **Strong decidability results:** GNF is used in proofs involving derivation bounds and CFL properties.
-

Applications and importance

GNF is especially useful in:

- **Top-down parsing techniques**
- **Theoretical proofs** involving derivation length
- **Decidability and complexity arguments**
- Showing that **every CFL can be parsed deterministically in principle** (though not always efficiently)

Unlike CNF, which emphasizes binary structure, GNF emphasizes **derivational control and predictability**.

One-paragraph summary

Greibach Normal Form is a normal form for context-free grammars in which every production begins with a terminal symbol, ensuring that each derivation step produces immediate output. This structure prevents infinite non-productive derivations, aligns the number of derivation steps with the length of the generated string, and enables clean top-down parsing. Although conversion to GNF is complex, every context-free language can be represented in this form, making GNF a powerful theoretical tool for understanding the behavior and limitations of context-free grammars.

These normal forms help in simplifying parsing algorithms, proving theoretical properties, and implementing computational models. The conversion process involves several steps, including eliminating ϵ -Productions, Unit Productions, and Binary Productions.

Formal Difference Table

Aspect	Chomsky Normal Form (CNF)	Greibach Normal Form (GNF)
Full Name	Chomsky Normal Form	Greibach Normal Form
Main Goal	Structural simplicity	Predictable derivation
Allowed Rule Form	$A \rightarrow BC$ or $A \rightarrow a$	$A \rightarrow a\alpha$
Right Side Starts With	Non-terminals or terminal	Always a terminal
Max non-terminals on RHS	Exactly 2	Any number (after terminal)
Terminals Position	Only alone on RHS	Only at the start
ϵ -production	Only $S \rightarrow \epsilon$ (optional)	Only $S \rightarrow \epsilon$ (optional)
Unit productions	Not allowed	Not allowed
Left recursion	Allowed	Not allowed
Parse tree shape	Binary tree	Linear growth
Parsing style	Bottom-up	Top-down
Key Algorithm	CYK algorithm	Predictive parsing
Derivation steps	Variable	Exactly equals string length
Conversion difficulty	Moderate	Difficult

Normal Forms for Context-Free Grammars (CFGs)

1. What Does “Normal Form” Really Mean?

A **normal form** is a **restricted but equivalent representation** of a context-free grammar.

- “Restricted” → Only certain types of productions are allowed
- “Equivalent” → The **language does not change**

Normal forms do **not reduce power**.

They only **standardize structure**.

This is extremely important in theory, because:

- Free-form CFGs are **too flexible**
 - Algorithms and proofs need **uniform patterns**
-

2. Why Normal Forms Exist (Deeper Reason)

CFGs describe **recursive tree structures**.

But unrestricted recursion becomes difficult to:

- Analyze mathematically
- Parse algorithmically
- Prove properties about

Normal forms **control recursion** while preserving meaning.

Think of it as:

Forcing all grammars to “speak the same grammatical style”.

3. Types of Normal Forms in CFGs

There are many normal forms, but **two are fundamental**:

Normal Form

Focus

Chomsky Normal Form (CNF) Binary tree structure

Normal Form

Focus

Greibach Normal Form (GNF) Leftmost derivation control

Derivation Tree in Automata Theory

In automata theory, we use derivation trees or parse trees to derive language from grammars. In this chapter, we will see the concept of derivation trees in the context of context-free-grammars. Derivation tree is a fundamental tool in understanding and representing the structure of strings generated by context-free grammars.

What are Derivation Tree?

A Derivation Tree, also known as a **Parse Tree**, is a visual representation of the process by which a context-free grammar generates a particular string. It provides a hierarchical breakdown of the string. This illustrates the sequence of production rules applied to derive the string from the grammar's start symbol.

Derivation trees provide a method of visualizing the structure of strings generated by context-free grammars. These are essential in several cases including understanding the concepts CFG derivation, parsing in compiler design, etc. In this chapter, we explained the concepts in detail with step by step examples for a clear understanding.

Key Elements of a Derivation Tree

Before learning how to construct a derivation tree, let's understand their essential components:

- **Root Vertex** – The root vertex represents the starting point of the derivation process and is always labeled with the grammar's start symbol.
- **Vertices (Internal Nodes)** – These nodes represent the non-terminal symbols of the grammar, serving as intermediate stages in the derivation.
- **Leaves** – These nodes represent the terminal symbols of the grammar, forming the final string derived from the grammar. They can also be labeled with the empty symbol, ϵ , if the grammar allows for empty productions.

Example of Context-Free Grammar

Let's consider the following context-free grammar **G** –

$$G=(V,T,P,S)$$

Where,

- $V = \{S, A, B\}$ (Variables or Non-terminal symbols)
- $T = \{0, 1\}$ (Terminal symbols)
- $P = \{S \rightarrow 0B, A \rightarrow 1AA \mid \epsilon, B \rightarrow 0AA\}$ (Production Rules)
- $S = S$ (Start Symbol)

This grammar defines a language where strings begin with a '0', then followed by a combination of '0's and '1's.

Derivation Tree Construction

To illustrate the formation of a derivation tree, let us examine the derivation of the string "001" from the grammar G:

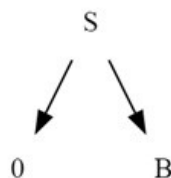
Root Vertex

We begin by placing the start symbol 'S' as the root vertex of the tree.

S

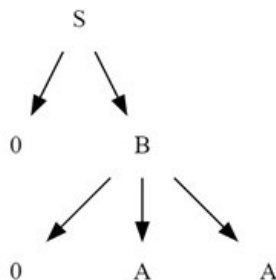
Applying Production Rules

Looking at the production rules, we see that 'S' can be replaced by "0B".



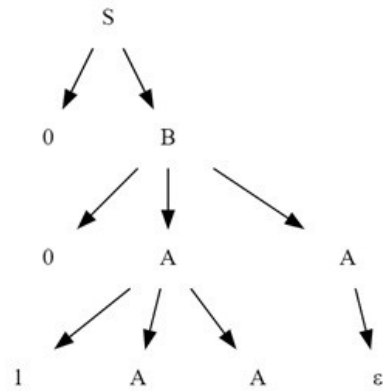
Continuing the Derivation

The rightmost vertex is 'B', and the production rule for 'B' is "0AA". Applying this rule, we obtain –

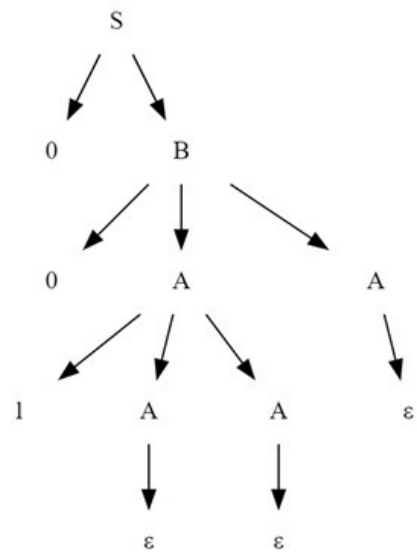


Reaching Terminal Symbols

Now, we have two 'A' variables. Since the production rule for 'A' allows for the empty string ' ϵ ', we can replace the right most 'A's with ' ϵ ', and left A to 1AA.



Now from the final non-terminals A to ϵ to get final tree.



This final structure represents the derivation tree for the string "001" from grammar G.

Left Derivation Tree and Right Derivation Tree

There are two primary methods for constructing derivation trees: The left derivation and right derivation. These methods dictate the order in which production rules are applied to non-terminal symbols within the sentential form.

- **Left Derivation Tree** – A Left Derivation Tree is generated by consistently applying production rules to the leftmost variable in each step of the derivation. This method uses expanding the non-terminal symbols on the left side of the sentential form.
- **Right Derivation Tree** – A Right Derivation Tree is obtained by applying production rules to the rightmost variable in each step. This method uses expanding non-terminal symbols on the right side of the sentential form.

Example of Left and Right Derivation Trees

Let's consider a new grammar and generate both left and right derivation trees to understand the difference in their construction.

Grammar –

$S \rightarrow aAS \mid aSS \mid \epsilon$

$A \rightarrow SbA \mid ba$

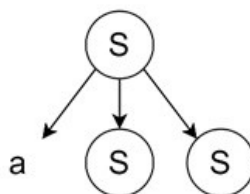
String to derive: "aabbbaa"

Left Derivation Tree

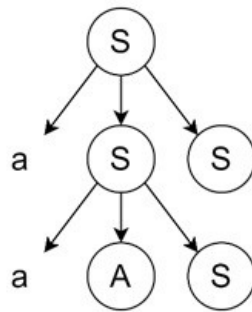
For the left derivation tree, we start from S.



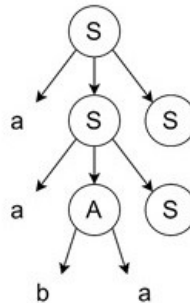
- **Applying Leftmost Derivation** – We apply the production rule ' $S \rightarrow aSS$ ' to the leftmost 'S'.



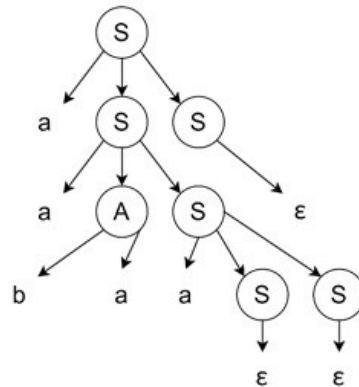
- **Expanding Leftmost Variable** – We expand the leftmost 'S' using the rule 'S $\rightarrow$ aAS'.



- **Continuing Leftmost Derivation** – We continue expanding the leftmost 'A' using the rule ' $A \rightarrow ba$ '.



- **Completing the Derivation** – We expand the next leftmost S to aSS, and from there remaining S to 'S $\rightarrow \epsilon$ '.

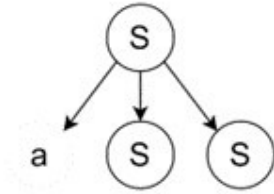


Right Derivation Tree

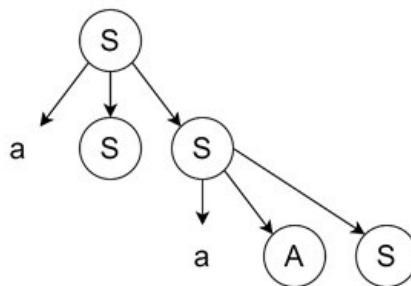
Like left most derivation, we start from the start symbol 'S'.



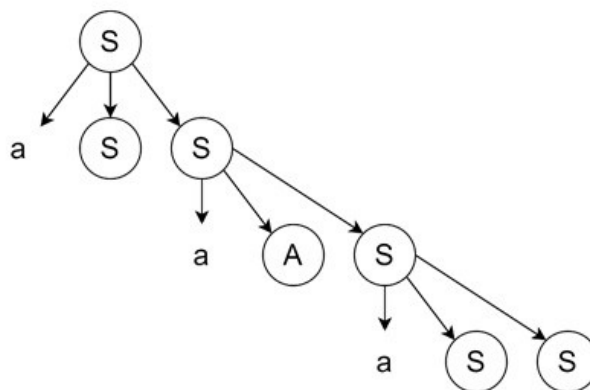
- **Applying Rightmost Derivation** – We apply the production rule ' $S \rightarrow aSS$ ' to the rightmost 'S'.



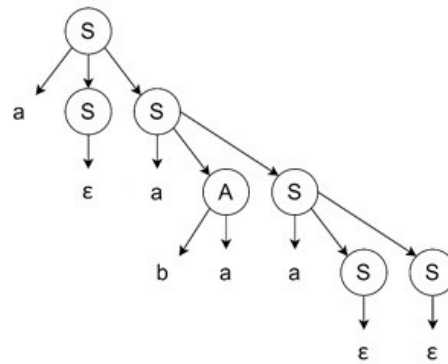
- **Expanding Rightmost Variable** – We expand the rightmost 'S' using the rule ' $S \rightarrow aAS$ '.



- **Continuing Rightmost Derivation** – We expand the rightmost 'S' using the rule ' $S \rightarrow aSS$ '.



- **Completing the Derivation** – We expand the remaining 'A' and 'S' from the right to left, the last two S will produce ϵ . Then A will bring 'ba' and the leftmost S will bring ϵ again.



What is the Pumping Lemma for CFLs?

The **Pumping Lemma for Context-Free Languages** is a **theoretical rule** that helps us **prove** that a language is **NOT** context-free.

☞ Very important:

- It is mainly a **negative test**
- It **cannot prove** a language *is* context-free
- It **can prove** a language *is not* context-free

The Pumping Lemma for Context-Free Languages says that if a language is truly context-free, then any sufficiently long string in it can be broken into five parts, where two parts can be repeated any number of times and the string will still belong to the language. This property comes from the tree-like structure of context-free grammars. If we can find even one long string where pumping always breaks the language rule, then the language is not context-free.

Why do we need it?

Regular languages have limited power, so they use a simple pumping lemma.

Context-Free Languages (CFLs) are **more powerful** (they can count, nest, match brackets, etc.), so their pumping rule is **more complex**.

CFLs are generated by **pushdown automata**, which use a **stack**.

Because of this stack behavior, CFL strings have a special **repeating structure**.

The Core Idea (Intuition)

If a language is **context-free**, then:

Any **long enough string** in the language must contain **some middle part** that can be **repeated (pumped)** and the string will **still remain in the language**.

If **no matter how you try**, pumping breaks the language rule, then the language is **NOT context-free**.

Formal Statement (Explained in Plain English)

If a language L is context-free, then:

1. There exists a number p (called pumping length)
2. Any string s in L with length $\geq p$

3. Can be divided into **five parts**:

$$s = u v x y z$$

such that:

Conditions (very important)

1. **v and y together are not empty**

- At least one of them must exist
- You must pump *something*

2. **Length of $vxy \leq p$**

- The pumped part is inside a limited window

3. **Pumping rule**

- For all $i \geq 0$:
- $u v^i x y^i z \in L$

Meaning:

- You can **repeat v and y the same number of times**
 - The string **must still belong to L**
-

Why five parts?

Because CFLs are based on **parse trees**.

When a parse tree is deep enough:

- Some variables repeat
- This repetition creates **two places** (v and y) where pumping happens
- x is the center part between them

This mirrors the **nested structure** of CFLs.

What does “pumping” really mean?

Think of it like this:

- Take a long string
- Identify **two matching expandable parts**
- Stretch or shrink them **equally**

- The grammar should still accept the string

If stretching breaks the rules → **not a CFL**

How the Pumping Lemma is Used (Strategy)

To prove a language is **NOT context-free**:

1. **Assume** the language *is* context-free
2. Then the pumping lemma must apply
3. Choose a **special string** from the language
4. Try **all possible ways** of splitting it into $u v x y z$
5. Show that **for some** i , the pumped string breaks the language rule
6. Contradiction → language is **not CFL**